

RIPHAH INTERNATIONAL UNIVERSITY



Database System
Spring 2026

Pharmacy Management System

Supervisor Name: Farwa Nawaz

Project Team Members

Name of Students	Sap ID	Program	Valid Email Address
Hafsa Saghir	62421	BSSE-4	hafsasaghir06@gmail.com
Mishal Asim	62516	BSSE-4	mishalasim234@gmail.com

Date of Submission
17/05/2026

Artifact-1

Introduction

1.1 Project Overview

The Pharmacy Management System (PMS) is a desktop-based application developed to automate and manage the daily operations of a pharmacy efficiently. The system is implemented using Java Swing for the graphical user interface and MySQL as the backend relational database. It provides a secure and user-friendly environment for managing medicines, users, billing records, and sales transactions. By digitizing pharmacy operations, the system reduces manual paperwork, minimizes human errors, and improves overall operational efficiency.

The application follows a role-based access control mechanism with two main user roles: Admin and Salesman. Each role is provided with a separate dashboard and restricted functionalities according to their responsibilities. The Admin manages user accounts and system administration tasks, while the Salesman handles medicine inventory and customer sales. All information is stored in a MySQL database named *pharmacy*, consisting mainly of the *appuser*, *medicine*, and *bill* tables.

1.2 Purpose and Scope

The main purpose of this project is to demonstrate the practical implementation of a database-driven desktop application for a real-world pharmacy environment. The project integrates database concepts with software development practices including design, implementation, testing, and documentation. The scope of the system covers the following major modules:

- **Authentication Module:** Provides secure login functionality for Admin and Salesman users with role-based access.
- **User Management Module:** Allows Admins to add, update, delete, and manage user accounts within the system.
- **Medicine Inventory Module:** Enables Salesmen to manage medicine records including adding, updating, viewing, and deleting medicines.
- **Sales and Billing Module:** Facilitates medicine selling operations, bill generation, and transaction management.
- **Profile Management Module:** Allows users to view and update their personal profile information stored in the database.

1.3 System Users

The Pharmacy Management System supports two categories of users:

Administrator (Admin)

The Admin acts as the superuser of the system and has complete control over user management functionalities. After successful login, the Admin is directed to the Admin Dashboard where they can add new users, update existing user details, delete users, and view all registered accounts. Admins can also update their own profile information. The *createdBy* field in the database stores the Admin responsible for creating user accounts, establishing a self-referencing relationship.

Salesman

The Salesman is responsible for handling medicine inventory and customer sales operations. After login, the Salesman accesses the Salesman Dashboard where they can add medicines, manage stock, process customer purchases, generate bills, and update their own profile. Salesmen are restricted from accessing user management functionalities.

1.4 Database Architecture

The backend database of the system is designed using MySQL and is named *pharmacy*. The database contains three core relational tables:

- **appuser:** Stores user details including username, password, role, and personal information. The username field is unique and the *createdBy* field references another Admin user.
- **medicine:** Stores medicine inventory records including medicine ID, name, company name, quantity, and price.
- **bill:** Stores billing transaction details such as bill ID, billing date, total amount, and the username of the Salesman who generated the bill.

The database design ensures proper normalisation, data consistency, and efficient retrieval of information through relationships and constraints.

1.5 Development Environment

The Pharmacy Management System was developed and tested using the following technologies and tools:

Component	Technology Used
Operating System	Windows 10 / Windows 11
Frontend Technology	Java Swing (JDK 17+)
Database	MySQL 8.x
Database Client	MySQL Workbench
Connectivity	JDBC
IDE	Apache NetBeans / IntelliJ IDEA
Hardware Requirement	Minimum 4 GB RAM

1.6 Objectives

The major objectives of the Pharmacy Management System are:

- To design and implement a normalised relational database for pharmacy management.

- To develop a fully functional desktop application using Java Swing and JDBC.
- To implement role-based access control for Admin and Salesman users.
- To perform CRUD operations efficiently on medicines, users, and billing records.
- To demonstrate database concepts such as primary keys, foreign keys, joins, aggregations, and subqueries.
- To test and validate all system functionalities through proper test cases and documentation.

Artifact-2

Requirement Anaylsis

Requirement Analysis:

Functional Requirements:

- The system shall provide a **login** screen where users enter their username and password. Based on the role stored in the database, the appropriate dashboard (Admin or Salesman) is displayed.

Admin Functionalities:

- Admins can register new users (Admin or Salesman) by providing name, date of birth, mobile number, email, username, password, and address.
- Admins can view all registered users in a tabular format. A user can be deleted by clicking on their row.
- Admins can search for a user by username and update their details including role, email, name, DOB, mobile number, and address.
- Admins can view and update their own profile information.

Salesman Functionalities:

- Salesmen can add new medicine records with Medicine ID, name, company name, quantity, and price per unit.
- Salesmen can view all available medicines in a table showing ID, name, company, quantity, and price. Medicines can be deleted by clicking a row.
- Salesmen can search medicine by ID and update its name, company, quantity (with an option to add quantity), and price.
- Salesmen can select a medicine from a dropdown, specify units to sell, and add items to a cart. A bill is generated and printed upon purchase.
- Salesmen can view all generated bills including Bill ID, date, total paid, and the salesman who generated it.
- Salesmen can view and update their own profile information.
- Both Admin and Salesman have the ability to **logout or exit** the application, with a confirmation dialog.

Non-Functional Requirements:

- **Security:** User passwords are stored in the database. Only authenticated users can access the system.
- **Performance:** The system shall load user and medicine data within acceptable response times for a local MySQL connection.

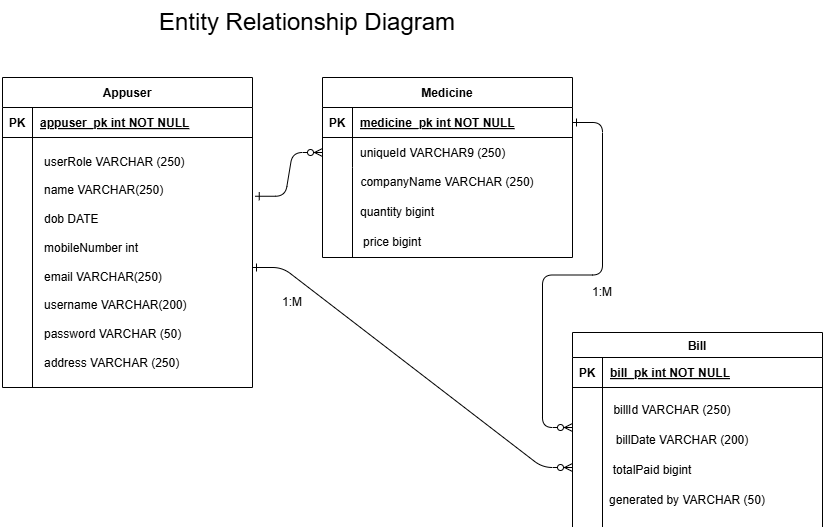
- **Reliability:** The system uses structured SQL queries with primary keys and proper relationships to ensure data integrity.
- **Usability:** The GUI is intuitive, with clearly labeled buttons, icons, and form fields for ease of use.
- **Maintainability:** The codebase is modular, separating the UI (Java Swing) from database logic (JDBC/MySQL).

Artifact-3

System Design

System Design

Entity Relationship Diagram



Relational Schema:

Here is the relational schema of this system:

- appuser (appuser_pk, userRole, name, dob, mobileNumber, email, username, password, address)
- medicine (medicine_pk, uniqueId, name, companyName, quantity, price)
- bill (bill_pk, billId, billDate, totalPaid, generatedBy)

Data Dictionary:

Table: appuser

Column	Data Type	Constraints	Description
appuser_pk	INT	PRIMARY KEY, AI	Auto-increment surrogate key
userRole	VARCHAR(50)	NOT NULL	Role: Admin or Salesman
name	VARCHAR(100)	NOT NULL	Full name of the user
dob	DATE		Date of birth
mobileNumber	VARCHAR(20)		Contact number
email	VARCHAR(100)		Email address
username	VARCHAR(50)	UNIQUE	Login username

Column	Data Type	Constraints	Description
password	VARCHAR(50)	NOT NULL	Login password
address	VARCHAR(200)		Home/work address

Table: medicine

Column	Data Type	Constraints	Description
medicine_pk	INT	PRIMARY KEY, AI	Auto-increment surrogate key
uniqueId	VARCHAR(20)	NOT NULL	Medicine identifier (e.g. 100)
name	VARCHAR(100)	NOT NULL	Medicine name
companyName	VARCHAR(100)		Manufacturing company
quantity	BIGINT		Available stock quantity
price	BIGINT		Price per unit (Rs.)

Table: bill

Column	Data Type	Constraints	Description
bill_pk	INT	PRIMARY KEY, AI	Auto-increment surrogate key
billId	VARCHAR(50)	NOT NULL	Unique bill identifier (e.g. Bill-62002...)
billDate	DATE		Date the bill was generated
totalPaid	BIGINT		Total amount paid (Rs.)
generatedBy	VARCHAR(50)		Username of the salesman

Artifact-4

Database Implementation

Database Implementation:

The following SQL statements were used to create the pharmacy database and its three tables in MySQL Workbench:

Create the database

```
CREATE DATABASE pharmacy;  
USE pharmacy;
```

Table: appuser

```
CREATE TABLE appuser (  
    appuser_pk INT PRIMARY KEY AUTO_INCREMENT,  
    userRole VARCHAR(50) NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    dob DATE,  
    mobileNumber VARCHAR(20),  
    email VARCHAR(200),  
    username VARCHAR(50) UNIQUE,  
    password VARCHAR(50) NOT NULL,  
    address VARCHAR(200)  
);
```

Table: medicine

```
CREATE TABLE medicine (  
    medicine_pk INT PRIMARY KEY AUTO_INCREMENT,  
    uniqueId VARCHAR(20) NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    companyName VARCHAR(100),  
    quantity BIGINT,  
    price BIGINT  
);
```

Table: bill

```
CREATE TABLE bill (  
    bill_pk INT PRIMARY KEY AUTO_INCREMENT,  
    billId VARCHAR(50) NOT NULL,  
    billDate DATE,  
    totalPaid BIGINT,  
    generatedBy VARCHAR(50)  
);
```

Artifact-5

SQL Queries

SQL Queries:

5.1 SELECT Queries

Retrieve all users

```
SELECT * FROM pharmacy.appuser;
```

Retrieve all medicines

```
SELECT * FROM pharmacy.medicine;
```

Retrieve all bills

```
SELECT * FROM pharmacy.bill;
```

Login authentication

```
SELECT * FROM pharmacy.appuser WHERE username='admin' AND password='admin';
```

Search user by username (for update)

```
SELECT * FROM pharmacy.appuser WHERE username='ali';
```

Search medicine by ID (for update)

```
SELECT * FROM pharmacy.medicine WHERE uniqueId='101';
```

5.2 INSERT Queries

Add a new user

```
INSERT INTO pharmacy.appuser (userRole, name, dob, mobileNumber, email, username, password, address) VALUES ('Salesman', 'Ali', '2026-02-09', '03495128382', 'ali09@gmail.com', 'ali', '12345', 'Islamabad');
```

Add a new medicine

```
INSERT INTO pharmacy.medicine (uniqueId, name, companyName, quantity, price) VALUES ('100', 'Panadol', 'testing', 2, 50);
```

Record a new bill

```
INSERT INTO pharmacy.bill (billId, billDate, totalPaid, generatedBy) VALUES ('Bill-6200277326100', '2026-03-27', 50, 'hafii');
```

5.3 UPDATE Queries

Update user profile (e.g. admin updates a salesman)

```
UPDATE pharmacy.appuser SET userRole='Salesman', email='ali09@gmail.com', name='Ali', mobileNumber='03495128382', address='Islamabad' WHERE username='ali';
```

Update medicine record

```
UPDATE pharmacy.medicine SET name='Brufein', companyName='Testing name', quantity=10, price=150  
WHERE uniqueId='101';
```

5.4 DELETE Queries

Delete a user by primary key

```
DELETE FROM pharmacy.appuser WHERE appuser_pk = 11;
```

Delete a medicine by primary key

```
DELETE FROM pharmacy.medicine WHERE medicine_pk = 2;
```

5.5 JOIN Query

Join bills with the salesman who generated them

```
SELECT b.billId, b.billDate, b.totalPaid, a.name AS salesmanName FROM pharmacy.bill b JOIN  
pharmacy.appuser a ON b.generatedBy = a.username;
```

5.6 GROUP BY and Aggregate Query

Total sales per salesman

```
SELECT generatedBy, COUNT(*) AS totalBills, SUM(totalPaid) AS totalRevenue FROM pharmacy.bill GROUP  
BY generatedBy;
```

5.7 Subquery

Find medicines priced above average

```
SELECT name, price FROM pharmacy.medicine WHERE price > (SELECT AVG(price) FROM  
pharmacy.medicine);
```


Artifact-6

User Interfaces

User Interfaces:

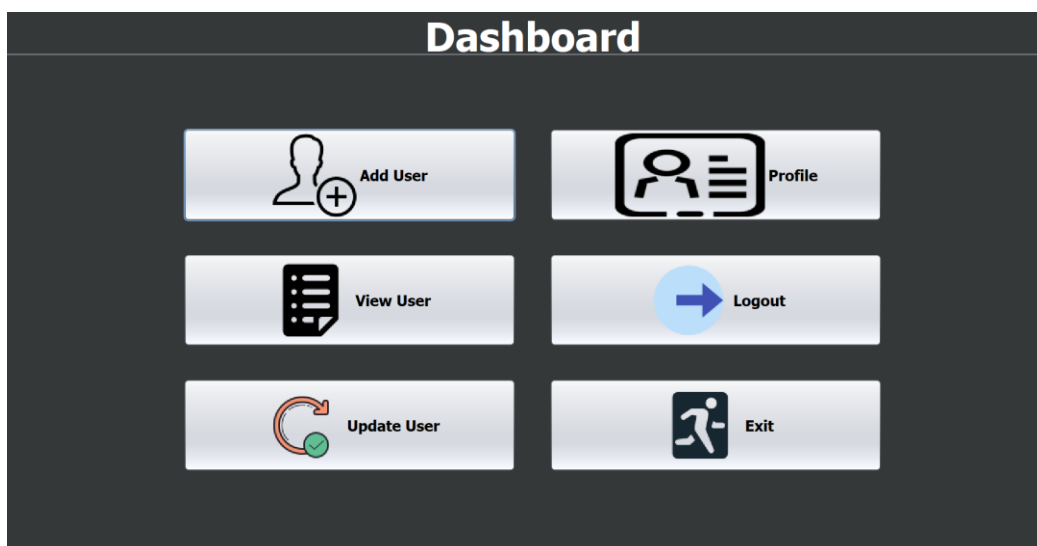
Login Page:

The login screen is the entry point of the application. The user enters their username and password, and the system validates credentials against the appuser table. Depending on the userRole, either the Admin Dashboard or Salesman Dashboard is displayed.



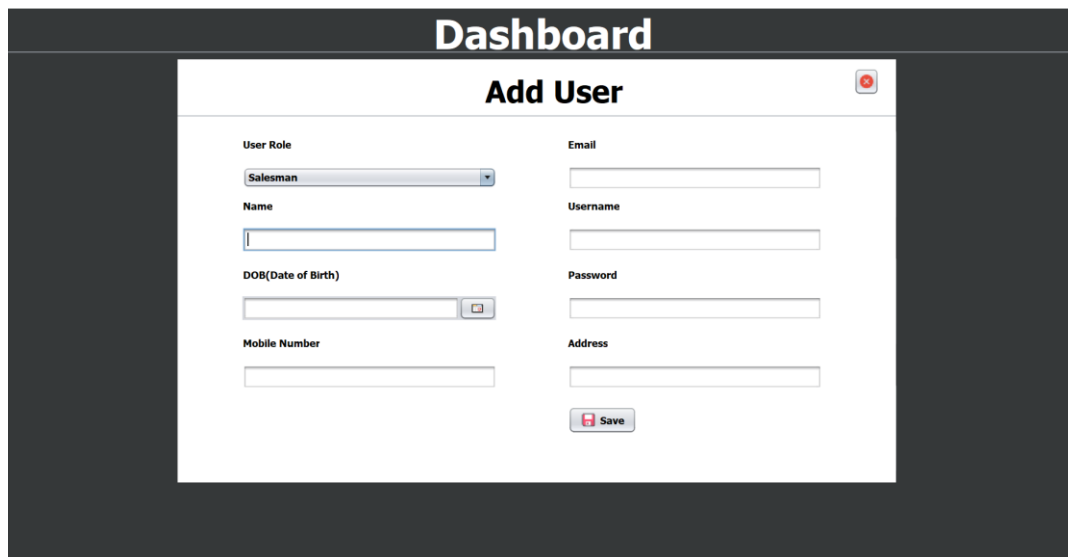
Admin Dashboard

The Admin Dashboard provides buttons for: Add User, View User, Update User, Profile, Logout, and Exit. An exit confirmation dialog is shown when the Exit button is clicked.



Add user:

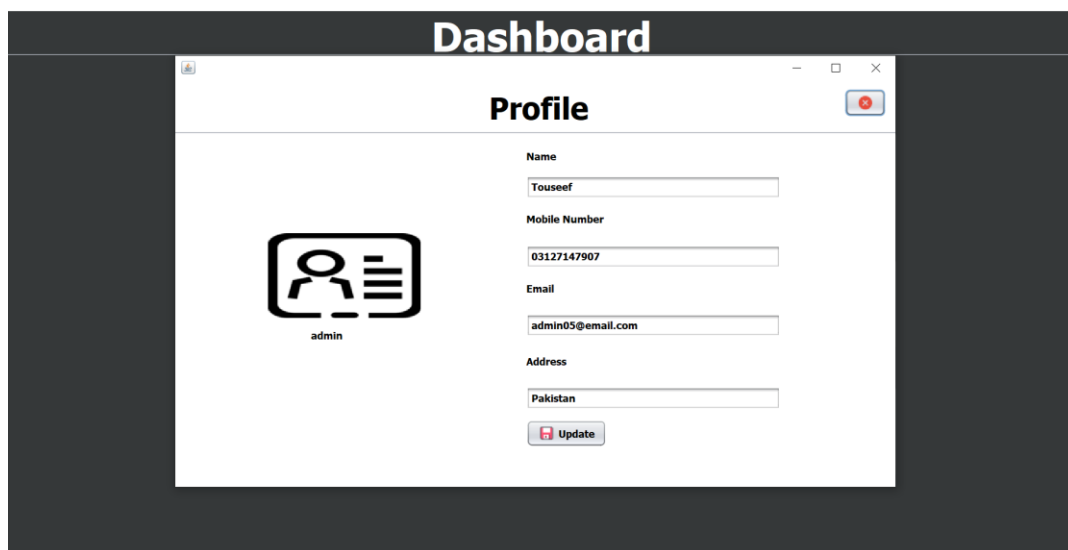
The Add User form allows the admin to register a new user. Fields include: User Role (dropdown), Name, Email, Username, DOB (date picker), Password, Mobile Number, and Address. Clicking Save inserts the record into the appuser table.



The screenshot shows a web application with a dark grey sidebar and a white main content area. The sidebar has a 'Dashboard' header. The main content area contains a form titled 'Add User' with a close button in the top right corner. The form is organized into two columns. The left column contains: 'User Role' (a dropdown menu with 'Salesman' selected), 'Name' (a text input field), 'DOB(Date of Birth)' (a date picker), and 'Mobile Number' (a text input field). The right column contains: 'Email' (a text input field), 'Username' (a text input field), 'Password' (a text input field), and 'Address' (a text input field). At the bottom center of the form is a 'Save' button with a floppy disk icon.

Profile View:

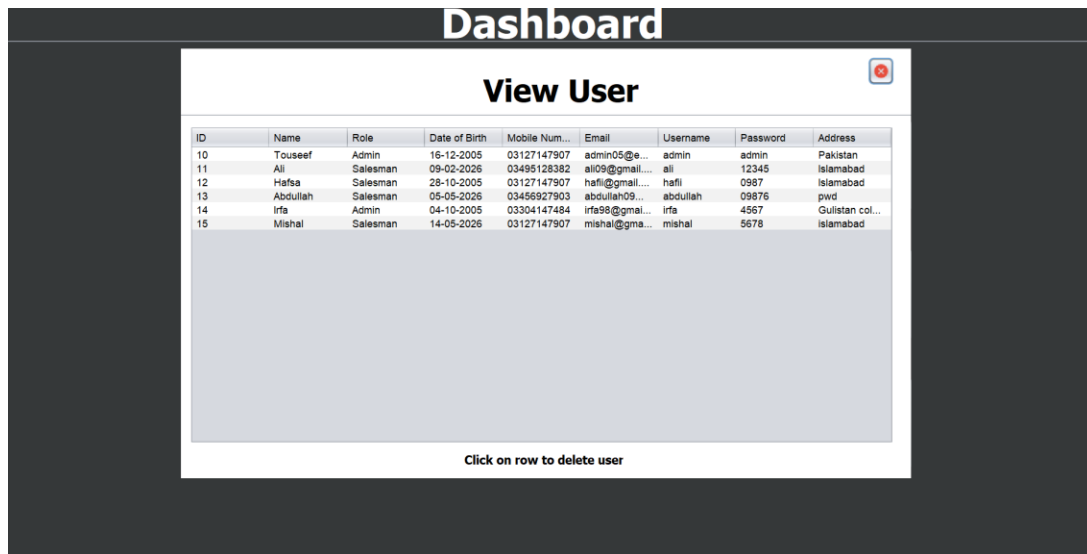
The Admin Profile screen displays the logged-in admin's name, mobile number, email, and address. The Update button saves any changes to the database.



The screenshot shows the same web application as the first image. The sidebar has a 'Dashboard' header. The main content area contains a form titled 'Profile' with a close button in the top right corner. On the left side of the form is a user icon (a person silhouette) with the text 'admin' below it. On the right side, there are four text input fields: 'Name' (containing 'Touseef'), 'Mobile Number' (containing '03127147907'), 'Email' (containing 'admin05@email.com'), and 'Address' (containing 'Pakistan'). At the bottom center is an 'Update' button with a floppy disk icon.

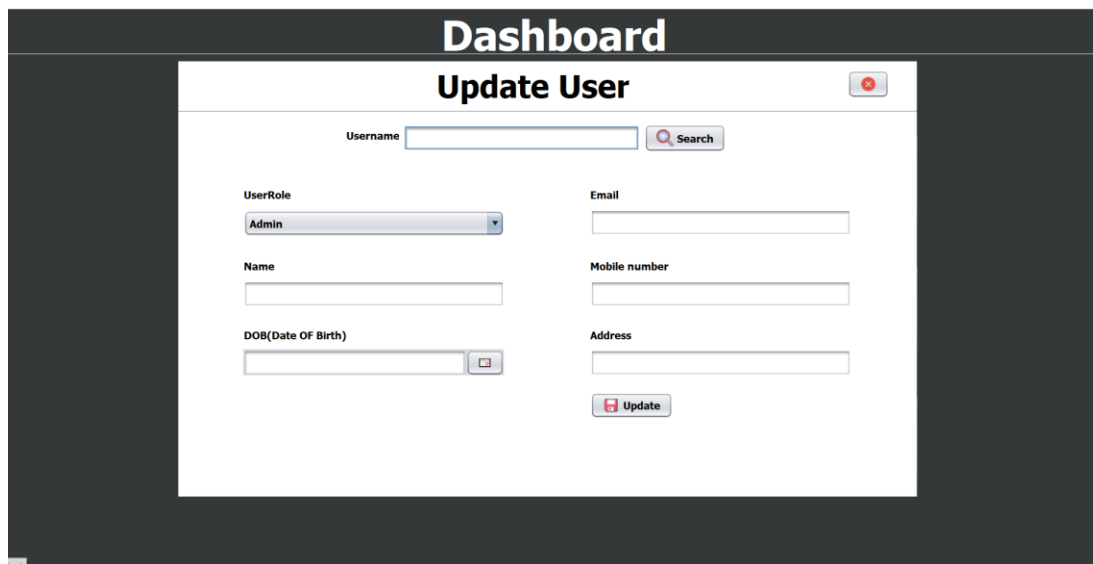
View User:

The View User screen displays all registered users in a table with columns: ID, Name, Role, Date of Birth, Mobile Number, Email, Username, Password, and Address. Clicking a row deletes that user from the database.



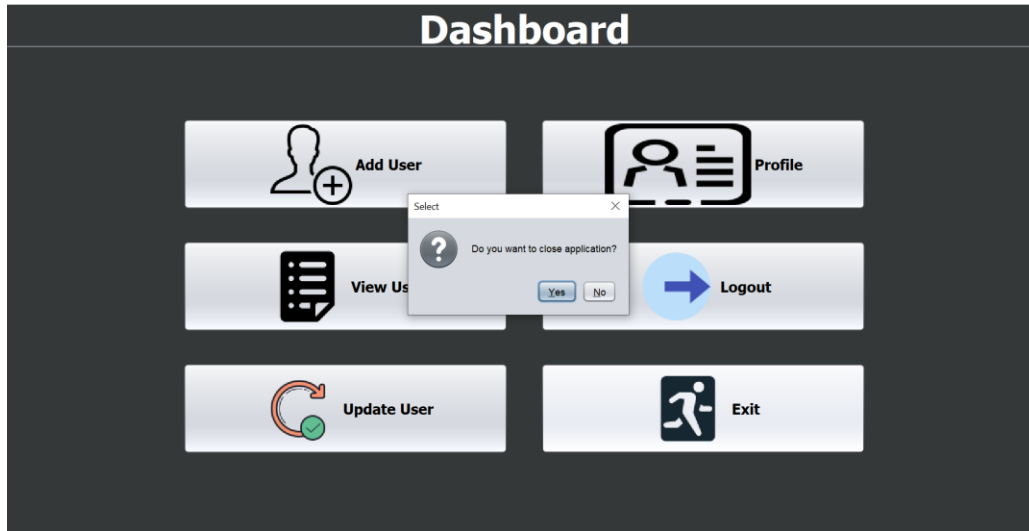
Update User:

The Update User screen allows the admin to search for a user by username and modify their details. All fields are pre-populated upon search, and clicking Update saves the changes.



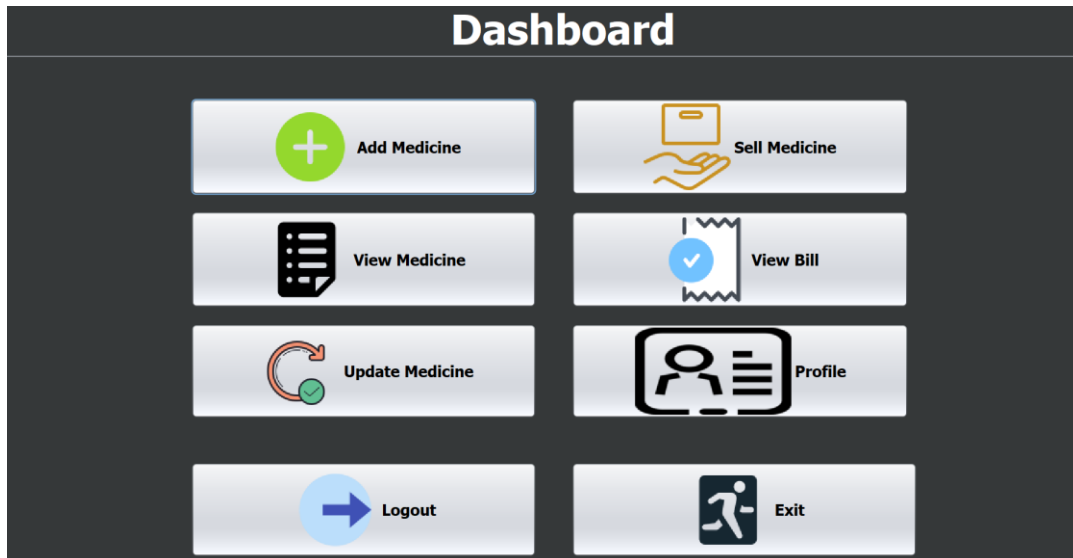
Logout Admin Dashboard:

The logout buttons allows user to logout from the admin dashbaord.



Salesman Dashboard

The Salesman Dashboard provides access to: Add Medicine, Sell Medicine, View Medicine, View Bill, Update Medicine, Profile, Logout, and Exit.



Add medicine:

The Add Medicine form captures: Medicine ID, Name, Company Name, Quantity, and Price Per Unit. Clicking Save inserts the record into the medicine table.

The screenshot shows a 'Dashboard' window with a central 'Add Medicine' form. The form has five input fields: 'Medicine ID', 'Price Per Unit', 'Name', 'Company name', and 'Quantity'. A 'Save' button is located to the right of the 'Name' field. At the bottom of the dashboard, there are two buttons: 'Logout' with a blue arrow icon and 'Exit' with a person icon.

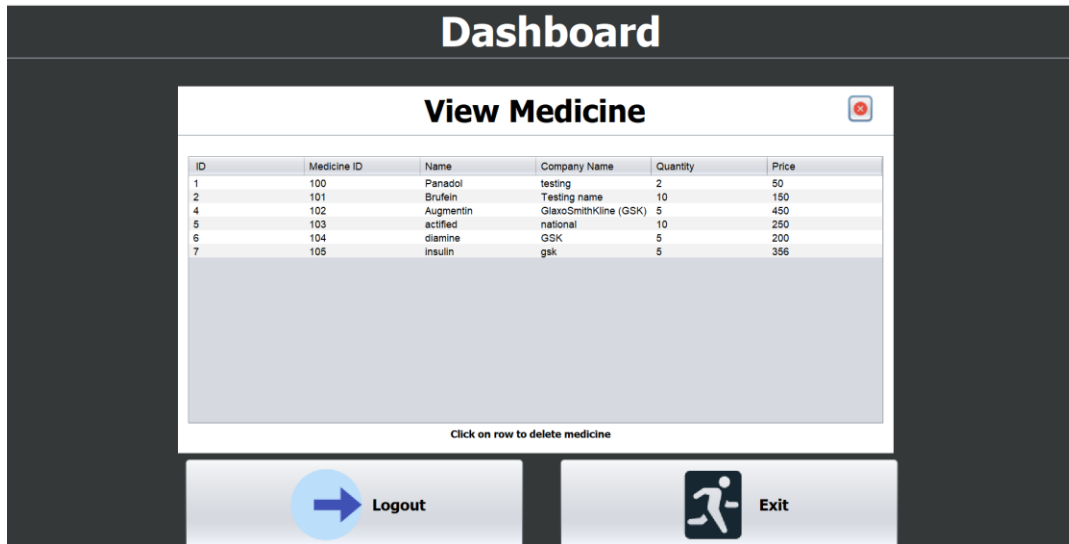
Sell Medicine:

The Sell Medicine screen has a searchable medicine list on the left. Selecting a medicine populates its details (ID, Name, Company Name, Price Per Unit). The salesman enters the number of units, and the total price is computed automatically. Items are added to a cart table, and clicking Purchase & Print completes the sale and generates a bill entry in the database and form pdf.

The screenshot shows a 'Sell Medicine' screen. On the left is a 'Search' section with a list of medicines: 100-Panadol, 101-Brufen, 102-Augmentin, 103-actifed, 104-diamine, and 105-insulin. The main area contains form fields for 'Medicine ID', 'Price Per Unit', 'Name', 'No. of Units', 'Company Name', and 'Total Price'. An 'Add to Cart' button is located below the 'Total Price' field. At the bottom, there is a table with columns: Medicine Id, Name, Company Name, Price Per Unit, No. Of Units, and Total Price. Below the table, the text 'RS: 00' is displayed, and a 'Purchase & Print' button is at the bottom right.

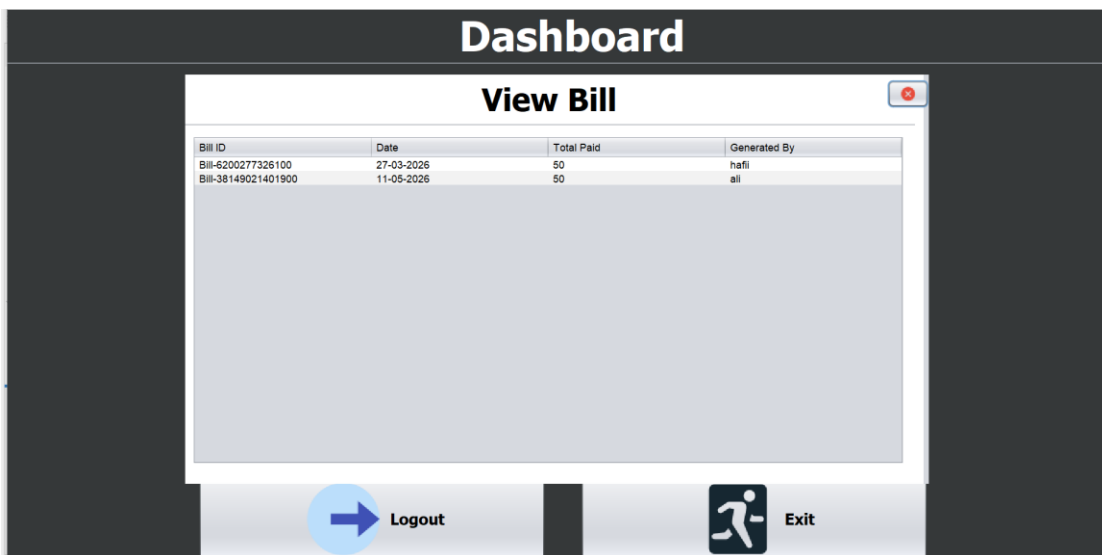
View Medicine:

The View Medicine screen shows all medicines in a table: ID, Medicine ID, Name, Company Name, Quantity, and Price. Clicking a row deletes the medicine from the database.



View Bill:

The View Bill screen displays all bills: Bill ID, Date, Total Paid, and Generated By (salesman username).



Artifact-7

Testing

Testing:

Test Case 01:

TEST CASE 1 — Valid Login

Module Name:	Login
Test Case ID:	PMS_Login_01
Tester Name:	Hafsa Saghir
Test Case Description:	To verify that the user can log in with correct username and password.
Prerequisites:	Program must be running. Login page must be displayed.
Environmental Information:	1. OS: Windows 10/11 2. System: Laptop 3. DB: MySQL (pharmacy)

Test Scenario	Checking that after entering a valid username and password the correct dashboard opens.
----------------------	---

Test Case ID	Test Steps	Test Input	Expected Results	Actual Results	Status	Comments
PMS_Login_01	Launch application	—	Login screen displayed	Login screen displayed	Pass	No issues found
	Enter Username	Username: admin	Field accepts input	Field accepts input	Pass	
	Enter Password	Password: admin	Field accepts input	Field accepts input	Pass	
	Click LOGIN button	—	Admin Dashboard opens	Admin Dashboard opened	Pass	No issues found

Test Case 02:

TEST CASE 2 — Invalid Login (Wrong Password)

Module Name:	Login
Test Case ID:	PMS_Login_02
Tester Name:	Mishal Asim
Test Case Description:	To verify that login fails when an incorrect password is entered.
Prerequisites:	Program must be running. Login page must be displayed.
Environmental Information:	1. OS: Windows 10/11 2. System: Laptop 3. DB: MySQL (pharmacy)

Test Scenario	Checking that an incorrect password prevents access and an error is shown.
----------------------	--

Test Case ID	Test Steps	Test Input	Expected Results	Actual Results	Status	Comments
PMS_Login_02	Launch application	—	Login screen displayed	Login screen displayed	Pass	
	Enter Username	Username: admin	Field accepts input	Field accepts input	Pass	
	Enter wrong Password	Password: wrong123	Field accepts input	Field accepts input	Pass	
	Click LOGIN button	—	Error message shown; dashboard does NOT open	Error dialog displayed	Pass	No issues found

Test Case 03:

TEST CASE 3 — Add New User (Admin Module)

Module Name:	User Management
Test Case ID:	PMS_User_01
Tester Name:	Hafsa Saghir
Test Case Description:	To verify that the admin can add a new Salesman user successfully.
Prerequisites:	Admin must be logged in. Add User form must be accessible from Dashboard.
Environmental Information:	1. OS: Windows 10/11 2. System: Laptop 3. DB: MySQL (pharmacy)

Test Scenario	Checking that a new user record is inserted into the appuser table after the form is submitted.
---------------	---

Test Case ID	Test Steps	Test Input	Expected Results	Actual Results	Status	Comments
PMS_User_01	Click "Add User" on Admin Dashboard	—	Add User form opens	Form opened	Pass	
	Select User Role	Role: Salesman	Dropdown shows Salesman	Salesman selected	Pass	
	Fill all fields	Name: Mishal, DOB: 14-05-2026, Mobile: 03127147907, Email: mishal@gmail.com, Username: mishal, Password: 5678, Address: Islamabad	All fields filled correctly	All fields filled	Pass	

Test Case 04:

TEST CASE 4 — Add Medicine (Salesman Module)

Module Name:	Medicine Management
Test Case ID:	PMS_Med_01
Tester Name:	Hafsa Saghir, Mishal Asim
Test Case Description:	To verify that a salesman can add a new medicine record to the inventory.
Prerequisites:	Salesman must be logged in. Add Medicine form must be accessible from Salesman Dashboard.
Environmental Information:	1. OS: Windows 10/11 2. System: Laptop 3. DB: MySQL (pharmacy)

Test Scenario	Checking that a new medicine record is inserted into the medicine table when the salesman submits the Add Medicine form.
----------------------	--

Test Case ID	Test Steps	Test Input	Expected Results	Actual Results	Status	Comments
PMS_Med_01	Click "Add Medicine" on Salesman Dashboard	—	Add Medicine form opens	Form opened	Pass	
	Enter Medicine ID	Medicine ID: 105	Field accepts input	Input accepted	Pass	
	Fill remaining fields	Name: insulin, Company: gsk, Quantity: 5, Price: 356	All fields filled correctly	All fields filled	Pass	
	Click Save button	—	Record inserted in medicine table; success message	Medicine saved	Pass	No issues found
	Verify via View Medicine	—	insulin appears in medicine list with correct data	Medicine visible in table	Pass	

Test Case 05:

TEST CASE 5 — Sell Medicine and Generate Bill

Module Name:	Sales & Billing
Test Case ID:	PMS_Bill_01
Tester Name:	Hafsa Saghir , Mishal Asim
Test Case Description:	To verify that a salesman can sell a medicine and a bill record is generated in the database.
Prerequisites:	Salesman must be logged in. At least one medicine must exist in the inventory.
Environmental Information:	1. OS: Windows 10/11 2. System: Laptop 3. DB: MySQL (pharmacy)

Test Scenario	Checking that selecting a medicine, adding it to cart, and clicking Purchase & Print creates a record in the bill table.
----------------------	--

Test Case ID	Test Steps	Test Input	Expected Results	Actual Results	Status	Comments
PMS_Bill_01	Click "Sell Medicine" on Salesman Dashboard	—	Sell Medicine screen opens with medicine list	Screen opened with medicine list	Pass	
	Select medicine from list	Medicine: 100-Panadol	Details auto-filled (Name, Company, Price)	Fields populated correctly	Pass	
	Enter number of units	No. of Units: 1	Total Price computed ($1 \times 50 = \text{Rs. } 50$)	Total Price: Rs. 50	Pass	
	Click "Add to Cart"	—	Item added to cart table below	Item appears in cart	Pass	
	Click "Purchase & Print"	—	Bill generated; new row in bill table; bill printed/displayed	Bill inserted; bill visible in View Bill	Pass	No issues found

Artifact-8

Results & Conclusion

Results & Discussions:

The Pharmacy Management System was successfully developed and tested. The following outcomes were achieved:

- **Role-Based Access Control:** The login module successfully differentiates between Admin and Salesman roles by querying the `userRole` field in the `appuser` table. Each role is directed to a distinct dashboard with relevant features.
- **Complete CRUD for Users:** Admins can create, read, update, and delete user accounts. All operations interact directly with the MySQL `appuser` table via JDBC queries.
- **Complete CRUD for Medicines:** Salesmen can manage the medicine inventory. The update form includes an "Add Quantity" feature to support restocking without overwriting existing quantities.
- **Sales and Billing:** The Sell Medicine module computes total prices dynamically as items are added to the cart. Upon completing a purchase, a bill record is inserted into the bill table with a unique Bill ID, date, total, and salesman identifier.
- **Data Integrity:** Primary keys and auto-increment fields prevent duplicate records. The `generatedBy` column in the bill table gives the name of user that is logged in to the system.
- **User Interface:** The Java Swing GUI provides an intuitive experience with icon-based buttons, modal forms, and data tables. Exit and logout operations include confirmation dialogs.

Areas for future improvement include implementing password hashing, adding a dedicated `bill_items` table to track individual items per bill, and building reporting features such as daily or monthly sales summaries.

1. Conclusion:

The Pharmacy Management System successfully demonstrates the integration of a relational MySQL database with a Java Swing desktop application. The project met all defined objectives:

- A normalized MySQL database (pharmacy) was designed and implemented with three tables: `appuser`, `medicine`, and `bill`.
- Role-based access was enforced through login authentication, directing Admin and Salesman users to their respective dashboards.
- Full CRUD operations were implemented for both user management (Admin module) and medicine management (Salesman module).
- A sales transaction system was built, allowing salesmen to sell medicines, generate bills, and view billing history.

This project reinforced key database concepts including schema design, SQL query writing, data integrity through keys, and real-world integration of a database backend with an application frontend. The system forms a solid foundation that can be extended with additional features such as reporting, inventory alerts, and multi-branch support.